

บทที่ 1

บทนำ

1.1 ความสำคัญและที่มา

การพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์ (Component-Based Software Development หรือ CBSD) เป็นกระบวนการในการสร้างซอฟต์แวร์ โดยนำคอมโพเนนต์ที่มีอยู่มาใช้ (reuse) เพื่อลดเวลาและค่าใช้จ่ายในการพัฒนา

และการพัฒนาสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์ (Server-side component architecture) ยังช่วยให้การพัฒนาแอปพลิเคชันระดับเอ็นเตอร์ไพรส์ (Enterprise application) ทำได้ง่าย นอกจากนั้นยังมีความน่าเชื่อถือ (Reliable) , มีระบบรักษาความปลอดภัย (Security) ที่ถูกต้องมากขึ้น ทั้งยังช่วยลดความซับซ้อนเนื่องจากโครงสร้างของระบบ ในปัจจุบันได้มีบริษัทต่างๆ ได้พัฒนาสถาปัตยกรรมที่ทำงานบนฝั่งเซิร์ฟเวอร์ (Server-side architecture) 2 สถาปัตยกรรมที่ได้รับการยอมรับมากที่สุดคือ

- Microsoft's Distributed interNet Applications Architecture (DNA) เป็นสถาปัตยกรรมของบริษัทไมโครซอฟท์ (Microsoft) ที่นำเสนอสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์คือ COM+ (Component Object Model plus)
- Sun Microsystems's Java 2 Platform, Enterprise Edition (J2EE) เป็นสถาปัตยกรรมของบริษัท ไมโครซิสเต็มส์ (Sun Microsystems) ที่นำเสนอสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์คือ EJB (Enterprise JavaBeans)

สถาปัตยกรรมของทั้ง 2 บริษัทมีความแตกต่างที่น่าสนใจคือ J2EE นำเสนอความสามารถในการทำงานข้ามแพลตฟอร์ม (portability) และ DNA นำเสนอความสามารถในการทำงานข้ามภาษา (interoperability) เนื่องจากทางบริษัท ไมโครซิสเต็มส์ได้ออกแบบภาษาจาวา (Java) ให้สามารถทำงานข้ามแพลตฟอร์ม การพัฒนาคอมโพเนนต์ด้วยภาษาจาวาทำให้คอมโพเนนต์มีความสามารถนี้อยู่ด้วย ในขณะที่ทางบริษัทไมโครซอฟท์ได้สนับสนุนการสร้างคอมโพเนนต์จากภาษาใดก็ได้เช่น Visual Basic และ Visual C++ คอมโพเนนต์ที่สร้างด้วย Visual Basic สามารถนำมาใช้กับ Visual C++ ได้ ความแตกต่างอีกอย่างหนึ่งคือ J2EE เป็น specification เท่านั้น แอปพลิเคชันเซิร์ฟเวอร์ (Application Server) จะถูกพัฒนาโดยผู้ผลิตรายอื่น เช่น BEA WebLogic Server , IBM WebSphere , Oracle 9i Application Server , SilverStream Application Server , iPlanet , Sybase Enterprise Application Server , Borland AppServer เป็นต้น

ทั้ง 2 สถาปัตยกรรมนี้มีข้อจำกัดที่ไม่สามารถนำกลับมาใช้ใหม่ (reuse) ข้ามสถาปัตยกรรมได้ แต่ภายหลังการเกิดของ XML , SOAP และแนวคิดของ Web Service ทำให้สามารถทำงานร่วมกันได้ผ่านทาง SOAP ซึ่งเป็นโปรโตคอลที่ใช้สำหรับ Remote Procedure Call (RPC) ผ่าน HTTP โปรโตคอล

1.2 วัตถุประสงค์ของโครงการ

ภายในโครงการนี้ได้ทำการศึกษาถึงทฤษฎี การสร้าง และการนำ เว็บเซอร์วิสไปใช้งานโดยมีจุดประสงค์ดังนี้

1. ศึกษาทฤษฎีพื้นฐานของคอมโพเนนต์
2. เพื่อศึกษาทำความเข้าใจ j2EE ของ Java
3. ศึกษาเรื่องเว็บเซอร์วิส และเทคโนโลยีที่เกี่ยวข้อง ได้แก่ SOAP, XML, WSDL, UDDI เป็นต้น
4. ศึกษาการสร้างเว็บเซอร์วิส โดยใช้ Enterprise Java Bean และ Bea Weblogic 7.0 ลงลึกในเทคโนโลยีของเว็บเซอร์วิส เช่น ทรานแซกชัน , การจัดการเซต , ความปลอดภัยในการเรียกใช้เว็บเซอร์วิส
5. ศึกษาวิธีการใช้งาน Bea Weblogic 7.0 เพื่อใช้สร้าง เว็บเซอร์วิส(Web service)
6. ทำการขยายการรองรับการใช้งานของระบบ (Scalability) โดยใช้เทคโนโลยี JMS
7. ศึกษาการเรียกใช้เว็บเซอร์วิสข้ามค่ายรวมทั้งเปรียบเทียบเทคโนโลยีของทั้งสองค่าย.NET และ จา
วา

1.3 ขอบเขตของโครงการ

1. ออกแบบและสร้างแอปพลิเคชันโดยใช้สถาปัตยกรรม แบบ 3 เทียร์ ร่วมกับการใช้หลักการพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์ด้วย Enterprise Java Bean
2. ใช้ทฤษฎีของ JMS เพื่อเข้ามาเพิ่มขนาด (Scalability) ให้กับแอปพลิเคชัน
3. สร้างแอปพลิเคชันทำการเรียกใช้เว็บเซอร์วิสคอมโพเนนต์ข้ามแพลตฟอร์มได้
4. ระบบที่พัฒนาเป็นระบบอีคอมเมิร์ซที่เกี่ยวกับ การจัดการแต่งงาน (Wedding Planner) โดยใช้ Enterprise Java Bean โดยกลุ่มจะทำการเป็นเว็บเซอร์วิสที่เป็นส่วนกลางโดยจะมีการ ติดต่อกับเว็บเซอร์วิสของแต่ละค่ายให้ทำงานโดยนำผลลัพธ์ที่ได้จากแต่ละเว็บเซอร์วิส ซึ่งได้แก่ บริษัททัวร์(Tour),บริษัทเอ็นเตอร์เทนเมนต์(Entertainment),ร้านถ่ายรูป(Photo Studio),ร้านขายของชำราย(Gift Shop) และ โรงแรม(Hotel) มาจัดการวางแผนการแต่งงานอีกครั้งหนึ่ง

1.4 วิธีการดำเนินงานของโครงการ

1. ศึกษาทฤษฎีพื้นฐานของ Enterprise Java Bean
2. ศึกษาทฤษฎีของ XML และ SOAP และส่วนประกอบในเว็บเซอร์วิส
3. ศึกษาการพัฒนาเว็บเซอร์วิสใน Bea Weblogic 7.0
4. ออกแบบระบบแอปพลิเคชันในลักษณะงานเชิงคอมโพเนนต์
5. ศึกษาการทำงานของ JMS
6. ศึกษาการปรับแต่งเพื่อให้เว็บเซอร์วิสมีความปลอดภัยในการสื่อสาร
7. ศึกษาการใช้งานร่วมกันรวมทั้งเปรียบเทียบเทคโนโลยีจากทั้งสองค่าย .NET และ จา
วา